

1. FCFS Scheduling algorithm

```
knsat@SRITHIKA: ~  
GNU nano 8.7.1  
#include <stdio.h>  
  
int main() {  
    int n, i;  
    int bt[20], wt[20], tat[20];  
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    printf("Enter burst time for each process:\n");  
    for(i=0; i<n; i++) {  
        printf("p%d: ", i+1);  
        scanf("%d", &bt[i]);  
    }  
  
    // waiting time  
    wt[0] = 0;  
    for (i=1; i<n; i++) {  
        wt[i] = wt[i-1] + bt[i-1];  
    }  
  
    // turnaround time  
    for (i=0; i<n; i++) {  
        tat[i] = wt[i] + bt[i];  
    }  
  
    printf("\nProcess\tBT\tWT\tTAT\n");  
    for(i=0; i<n; i++) {  
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);  
    }  
  
    return 0;  
}
```

```
knsat@SRITHIKA:~$ nano fcfs.c  
knsat@SRITHIKA:~$ gcc fcfs.c -o fcfs  
knsat@SRITHIKA:~$ ./fcfs  
Enter number of processes: 6  
Enter burst time for each process:  
p1: 2  
p2: 4  
p3: 6  
p4: 8  
p5: 10  
p6: 12  
  
Process BT      WT      TAT  
P1      2        0        2  
P2      4        2        6  
P3      6        6        12  
P4      8        12       20  
P5     10       20       30  
P6     12       30       42  
knsat@SRITHIKA:~$ gcc fcfs.c -o fcfs  
knsat@SRITHIKA:~$ ./fcfs  
Enter number of processes: 1  
Enter burst time for each process:  
p1: 14  
  
Process BT      WT      TAT  
P1     14        0       14
```

2. SJF_NONpreemptive

```
knsat@SRITHIKA: ~  
GNU nano 8.7.1  
#include <stdio.h>  
  
int main() {  
    int n, i, j;  
    int bt[20], wt[20], tat[20];  
    int temp;  
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    printf("Enter burst time:\n");  
    for(i = 0; i < n; i++) {  
        printf("p%d: ", i + 1);  
        scanf("%d", &bt[i]);  
    }  
  
    // Sort burst times (Shortest Job First)  
    for (i = 0; i < n - 1; i++) {  
        for (j = i + 1; j < n; j++) {  
            if(bt[i] > bt[j]) {  
                temp = bt[i];  
                bt[i] = bt[j];  
                bt[j] = temp;  
            }  
        }  
    }  
  
    // Calculate waiting time  
    wt[0] = 0;  
    for (i = 1; i < n; i++) {  
        wt[i] = wt[i-1] + bt[i-1];  
    }  
  
    // Calculate turnaround time  
    for (i = 0; i < n; i++) {  
        tat[i] = wt[i] + bt[i];  
    }  
  
    // Display results  
    printf("\nProcess\tBT\tWT\tTAT\n");  
    for(i = 0; i < n; i++) {  
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);  
    }  
  
    return 0;  
}
```

```
knsat@SRITHIKA:~$ nano sjf.c  
knsat@SRITHIKA:~$ gcc sjf.c -o sjf  
knsat@SRITHIKA:~$ ./sjf  
Enter number of processes: 5  
Enter burst time:  
p1: 2  
p2: 9  
p3: 12  
p4: 14  
p5: 30  
  
Process BT      WT      TAT  
P1      2        0        2  
P2      9        2       11  
P3     12       11       23  
P4     14       23       37  
P5     30       37       67
```

3. Priority

```
knsat@SRITHIKA: ~
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time and priority:\n");

    for(i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority (Lower number = Higher priority)
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if(priority[i] > priority[j]) {
                // Swap priority
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                // Swap burst time along with priority
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Calculate Waiting time
    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }

    // Calculate Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Print results
    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, bt[i], priority[i], wt[i], tat[i]);
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano priority.c
knsat@SRITHIKA:~$ gcc priority.c -o priority
knsat@SRITHIKA:~$ ./priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 2
P1 Priority: 4
P2 Burst Time: 6
P2 Priority: 8
P3 Burst Time: 10
P3 Priority: 12
P4 Burst Time: 14
P4 Priority: 16
P5 Burst Time: 18
P5 Priority: 20

Process BT      Priority      WT      TAT
P1      2         4           0        2
P2      6         8           2        8
P3     10        12          8       18
P4     14        16         18       32
P5     18        20         32       50
```

4. ROUND ROBIN

```
knsat@SRITHIKA: ~
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin Scheduling
    while (1) {
        done = 1;
        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0; // There is still a pending process

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                } else {
                    time = time + rem_bt[i];
                    wt[i] = time - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }
        if (done == 1) {
            break;
        }
    }

    // Calculate Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Print results
    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

```
knsat@SRITHIKA: ~
knsat@SRITHIKA:~$ nano rr.c
knsat@SRITHIKA:~$ gcc rr.c -o rr
knsat@SRITHIKA:~$ ./rr
Enter number of processes: 5
Enter burst time:
P1: 14
P2: 30
P3: 26
P4: 27
P5: 28
Enter time quantum: 3

Process BT      WT      TAT
P1      14      48      62
P2      30      94      124
P3      26      89      115
P4      27      91      118
P5      28      97      125
```